

Preliminary

I hope that I will actually manage to have some runnable version of this project and not do the ADHD thing and abandon it halfway through. So keep in mind that this will be an evolving document and more tailored to keep me on track

Goal

For some reason I have the obsession on having an automatically controlled model railway with accurate dispatching. For this you need an intensive sensor arrangement, which can be quite cumbersome to install many block detectors. Additionally in a station application it is infeasible/expensive to have that many blocks. Point detection is another problem in that it is difficult to track direction and actual train identification via NFC is unreliable[verify]. This doesn't even mention the need to hide these detectors.

My intent is to have a model which can be deployed to a small device and will be capable of the following things:

- detection of wagons and their position along a track
- detection of consists of wagons(train) as an entity
- identification of individual wagons for shunting operations

This is a rough roadmap of feature completeness. To achieve this substations changes, such as multiple cameras, may be needed.

Pros and Cons of existing solutions

Here there will be a short summary of all positioning systems for model rail

Design considerations

In the current state of the project I will be happy on reliably positioning a single wagon. Much more important are the design constraints. I want to limit the computation power to a primitive SBC(PI Zero) or embedded controller(ESP CAM). I currently only have a ESP CAM so this will be the intended limit. The processing by the PC is not yet finalized. Therefore a simple text stream of `ID:[ID], pos:float`. This processor limit will constrain the size of the model and require special attention to downsizing it.

The choice is also to limit the application to a single static known camera. A single camera simplifies the data processing as no observations need to be aligned. While multiple cameras allow for 3D vision, trigonometry should be able to substitute it. Being a static known camera whose position and focal length is known positions in 2D space should be easily translated to a position on the base plate. To make this easier this project will limit itself to train track of the same height. An alternative is also the existence of homing points of known distance and size. This is to be investigated.

Existing tech

This section will be the literature review

Computer vision tasks can be subdivided into object detection, what object is in an image, or object segmentation, color in the surface of that object. Object detection in its most primitive form is unable to locate the subregion where the object is supposed to be. More modern variants such as YOLO are able to provide bounding boxes. Yet these bounding boxes suffer as they do not conform to the actual object. As getting the train's orientation a simple bounding box is not sufficient. Image segmentation instead tries to either separate all instances of a class from other classes, or in the panoptic variant to also separate individual class instances from another. But here again the problem of getting the orientation from the 2D image is difficult. In an ideal case with a perfect mask one could try to fit a diagonal into the resulting form. This is to be investigated on if OpenCV offers such out of the box tools. Current assumption is no.

Keypoint detection offers the benefit of an end-to-end detection of the relevant components of the train. The front and back in 2D space.

Training: Training is a special consideration due to two specific aspects. The camera is from a ca 30° elevation. This is a diversion from most training datasets such as COCO who have front focused images. This

may cause inaccuracies later on. The second is that there exists no data which I could find of (model) trains with their keypoints labeled. Special attention will therefore be placed on models which can learn these features in a self-supervised manner.

Models

I plan on scouring the web in this sequence to find a collection of suitable base architectures and backbones. A sufficiently trained backbone should boost detection accuracy by having learned from a highly varied dataset and will keep my training efforts low.

1. Shitty AI blogs
 - the offer a quick list of popular open source projects and the general consensus
2. Papers with Code
 - Benchmarks of the most popular models
3. actual research in the literature
 - oh god, help me :(

Approaches

There are two core methods on how to do multi object keypoint detection, as the challenge is no longer just to detect keypoints at all, but to also assign them to the same object. One can choose to first detect all relevant objects and afterwards for each individually in their cropped region the keypoints (top-down) or all keypoints are detected at the same time and in a second step they are assigned to objects (bottom-up). Examples of the detect-then-key method are This has the added challenge of requiring two models. A detector and a key point regressor. Though in such an application interference from other instances should be minimized and a simpler keypoint detection method can be used. For the key-then-cluster methods existing methods are [1] and .

The core structure of the network will be the self supervised video encoder-decoder method from [2]

A core challenge is the lack of any supervised training data. This was already given as a condition, but from that follows a much more severe impact on to the key point detection. Most, if not all, Keypoint detection (specifically pose estimation) methods make use of unique keypoints if doing a bottom up approach. This was the main reason why a top-down approach was pursued. But as noted in [3] occlusion of objects can be a major concern for such applications. But in this applications there can not be unique points as many trains can be symmetrical. A method to match unknown but unique points is therefore needed. Landmark detection is the field of trying to match multiple different images of the same object. This is somewhat different to this usecase where multiple different points need to be identified. In Superpoint [4] one such method is proposed where points are matched by minimizing a hinge loss. This description can then be utilized to create artificial point categories.

After the identification the other aspect of associating parts comes up. As I don't want to make a full segmentation model using such masks are not possible. an option is using some weak points to identify objects as some MASTER part. See GNN The idea is then to use an ideal labeled image or model as a guide in some way to associate the parts with the proper wagons, where the matching loss needs to be minimized.

Finetuning of the keypoint locator.

The model can be finetuned to only detect model train keypoints by biasing it using a train mask. This can be gotten by randomly sampling examples and placing keypoints of the in and out class like in [5]

Bibliography

- [1] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields."
- [2] T. Jakab, A. Gupta, H. Bilen, and A. Vedaldi, "Unsupervised Learning of Object Landmarks through Conditional Image Generation."

- [3] S. Kreiss, L. Bertoni, and A. Alahi, “OpenPifPaf: Composite Fields for Semantic Keypoint Detection and Spatio-Temporal Association.”
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description.”
- [5] B. Cheng, O. Parkhi, and A. Kirillov, “Pointly-Supervised Instance Segmentation.”